# Five AI sessions, one 2014 ilastik workflow: what happened

*GloBIAS 2026 · Group 5 · Session 06 "Benchmarking". Written for readers who have not watched an AI agent work before.*

## What an "agent session" is

Claude is a large language model. In a chat it only writes text. In an *agent session* it also gets a computer: it can run commands, install software, read and write files, read error messages and try again. It does this in a loop, on its own, until the task is done or it gets stuck. The person watches, answers questions and can step in at any time. Everything below happened in such sessions, in October 2026, with ilastik's MitoCheck example: a 53-frame movie of dividing cell nuclei, plus the two ilastik projects that were trained by hand in 2014, one for finding nuclei and one for tracking them.

## The workflow, in plain words

First, **pixel classification**: the 2014 project gives every pixel of every frame a score, "how nucleus-like is this pixel?", which makes a *probability map*. Then, **tracking**: the second project finds each nucleus in that map, follows it from frame to frame and notices divisions. A nucleus and all its descendants form one *lineage*. The goal was the final tracking labels, with ilastik run "headless", meaning from the command line with no window.

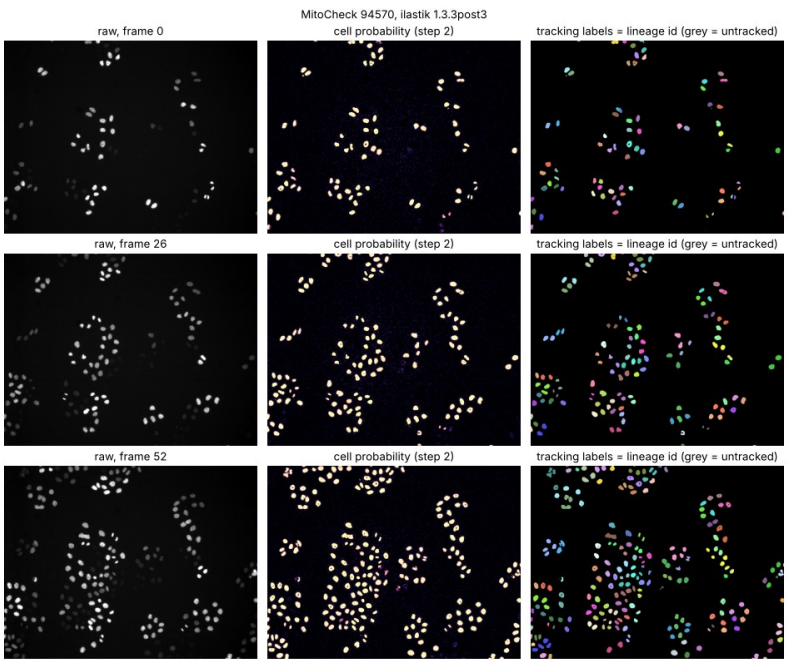



**Figure 1.** The workflow on three frames of the movie (0, 26, 52). Left: the raw frame. Middle: the probability map from pixel classification ("step 2" in the repository's script numbering), bright where the classifier sees a nucleus. Right: the tracking labels, each lineage in its own colour; grey objects were detected but not tracked. Output of ilastik 1.3.3post3 in Run 0.

### Run 0: the original (21 October, Opus)

The agent started with an almost empty code repository and no access to ilastik's website. Getting to the result took seven "walls", among them: the network block (the person had to allow the site); ilastik 1.4.1 no longer installs with the usual package manager, so it used a newer pre-release, 1.5.0a3; that version could not place the 2014 project's own training labels, because it reads them with rows and columns swapped compared with the 2014 file; and the tracking project kept its settings in a place that current versions no longer look. For each wall the agent read the error and, where needed, ilastik's own program code, then switched version, linked a file into place or wrote a small fix script (two of the walls got one), and tried again. When the person suggested the old version 1.3.1, the agent found it is not in the public archive, tried 1.3.2post1, which crashed because of the computer's system libraries rather than the project, and settled on 1.3.3post3. Both versions then gave the same counts: 7,485 detections over the 53 frames, 357 tracks in 117 lineages, 120 divisions; their label images differ in 161 of 3.6 million pixels. The session also made two claims it had not checked (that the versions numbered lineages differently; that 1.3.2post1 "crashes on this project"). A second look caught both, and the text was corrected.

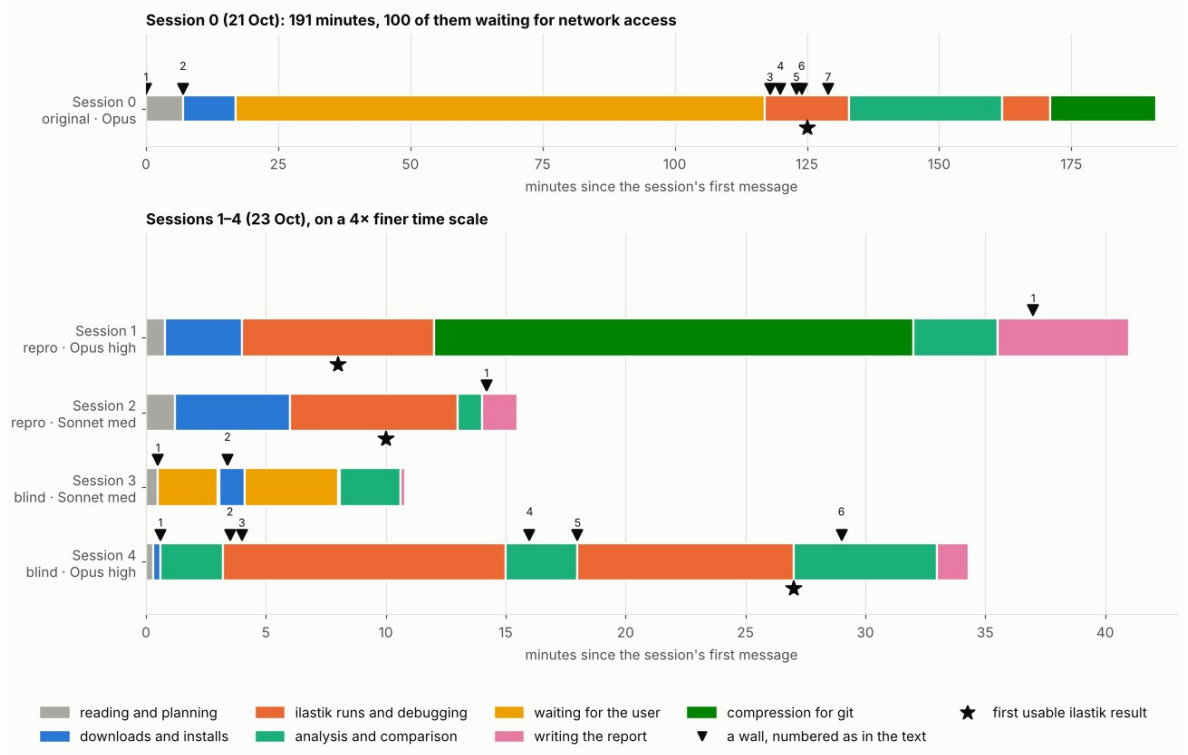



**Figure 2.** How each session spent its time, in minutes from its first message. Colours are activities; black triangles are the walls, numbered as in the benchmark's wall table; the star marks the first usable ilastik result. Run 0 is drawn on its own scale: 191 minutes, 100 of them waiting for network access. Run 3 never reaches a star; Run 4's star marks its first retrained classifier, not the example's tracking.

### Runs 1 and 2: reproduce it, with the notes (23 October)

Two fresh sessions, one Opus (high effort) and one Sonnet (medium), were pointed at Run 0's repository and told to regenerate the results, not copy them. Neither needed the person at all. Both read the documentation, re-downloaded the data, ran the committed scripts and compared their

output with Run 0's. Sonnet finished in about 15 minutes, Opus in 41, mostly because Opus also compressed the large files so they could be stored in the repository. In both reruns the 1.5.0a3 tracking labels were identical to Run 0's, pixel for pixel, and Opus showed the probability maps identical too. The 1.3.3post3 tracking was not: in the Opus rerun two lineages swapped numbers and 113 pixels changed object, 80 of them on the border between two touching nuclei; the Sonnet rerun differed in 50 pixels. So the pipeline is reproducible, but one step in the old version is not exactly repeatable from run to run.




**Figure 3.** Who hit which wall. Orange: the session hit it. Blue: Run 0's scripts, which Runs 1 and 2 could read, had already fixed it. Green: avoided another way. Grey: not met. The row-and-column wall (row 5) is the only ilastik wall two sessions hit unaided, Run 0 in ilastik 1.5.0a3 and Run 4 in 1.4.2.

## Runs 3 and 4: the same question, without the notes

Two more sessions got only the assignment PDF, with the group's question ("Can consensus become ground truth?"), and one line, verbatim: *"I want to answer group 5 task question, use ilastik and the mitocheck dataset"*. They could not see the repository. Neither ran the example's tracking. Each built a consensus experiment with several "annotators", which is what the question asks for, then compared voting rules. Their strategies parted in the first minute, and this section follows each in more detail.

### Run 3 (Sonnet, medium effort): 11 minutes, asked twice

1. It read the PDF and found the group's question.
2. It looked at the computer: no ilastik installed, some Python packages missing. From that it concluded that ilastik needs a graphical window and cannot run here. It never looked for a command-line version, although the download page it later fetched lists Linux installers. This one belief shaped everything after.

3. It guessed, saying the guess was unverified, that the data was on a public image archive (IDR), and asked the person three questions. The person pointed to ilastik's website. It then downloaded the 118 MB example.
4. It found one movie, three projects and no multiple annotators, and asked whether the group had any. The person said: no, run it with the existing labels.
5. It rebuilt the classifier with a general machine-learning library (scikit-learn), approximating the image measurements ilastik uses (smoothness, edges, blob-likeness, at two scales). The 2014 labels are stored in 27 small blocks; each of five annotators was a classifier of the same family as ilastik's, trained on a different random 16 of those 27. So all five are the same person's labels, resampled.
6. It compared five rules on seven frames: majority (3 of 5), unanimity (all 5), votes weighted by each annotator's accuracy, "expert adjudication" (ties broken by ilastik's own 2014 output) and "exclude the uncertain pixels". The five agreed closely (a standard agreement score, Fleiss' kappa, was 0.87–0.88, where 1 is perfect agreement); the share of disputed pixels grew from 1.9% in frame 0 to 4.8% in frame 52, as the field fills with cells. In frame 0, unanimity shrank the nucleus area by 4.4%; weighting changed nothing. Its conclusion: consensus "is necessary for measuring reliability, but not sufficient to define ground truth", because it cannot detect errors all annotators share. It flagged itself that its "expert" comparison was circular.

**Run 4 (Opus, high effort): 34 minutes, no questions**

1. It read the PDF, then went straight to ilastik's download page. Within the first minute it had started downloading the data and ilastik 1.4.2 for Linux, and from then on it worked with ilastik headless.
2. It opened the 2014 project, worked out that label 1 is background and label 2 is nucleus, and saw that the strokes are sparse: 5,157 pixels in total, almost all on frame 0.
3. It kept the real 2014 labels as annotator **A0** and wrote a program that invents five more. The program first finds nucleus candidates with a simple brightness cut-off, then paints strokes on them in a chosen style: **A1** tight edges (part of the rim labelled background), **A2** a two-pixel halo (part of the surroundings labelled nucleus), **A3** "dim nuclei are background", **A4** very few strokes (three nuclei on one frame, 922 pixels), **A5** late frames only. Each annotator became its own copy of the 2014 project, with the labels replaced and the old classifier removed, retrained headless.
4. Four walls, all fixed alone: missing Python packages; headless ilastik opens projects read-only; the 2014 label blocks were refused ("Rois do not intersect"); and the first outputs were nonsense, because its labels had landed in the image with rows and columns swapped. It diagnosed the last one with ilastik's own Python and rewrote the labels the other way round. This is the same trap Run 0 fell into. It also killed its own shell once and ran out of memory once.

5. It compared seven rules over the whole movie: union (anything any annotator marked), majority (4 of 6), unanimity (all 6), mean score (average the six classifiers' scores), weighted (A0 counted twice), STAPLE (a method that estimates how reliable each annotator is from their agreement alone, then trusts the reliable ones more) and a three-way version that leaves disputed pixels out.
6. It added a second experiment, a **biased pool**: A0 plus A1 and three "dim nuclei are background" annotators. Then it checked the nuclei the majority had rejected by another route: do they persist in the neighbouring frames?

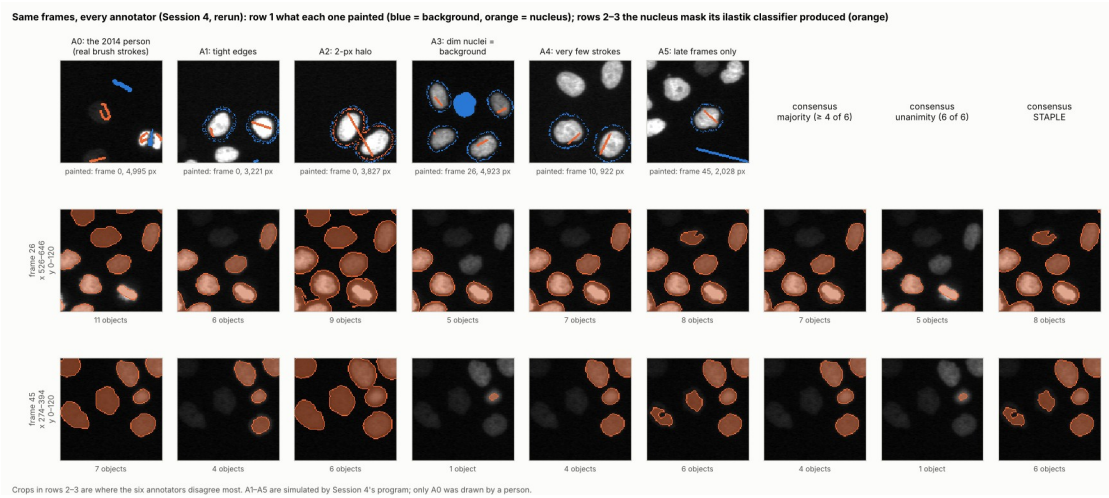



**Figure 4.** Run 4's annotators, drawn from a rerun of Run 4's own scripts. Row 1: what each annotator painted (blue = background, orange = nucleus), each on the frame it painted most. Rows 2–3: the same two 120 × 120 pixel crops of frames 26 and 45 for every annotator, with the nucleus mask its retrained classifier produced, and the three consensus rules. A1, A3 and A4 miss most of the dim nuclei at the left of each crop that A0 finds; majority vote drops them.

### What the two found

Run 4's main pool: A0, the classifier trained on the real 2014 strokes, finds 138 nuclei per frame; the simulated annotators 77 to 123. Which rule you choose then sets the count: union 124, majority 111, STAPLE 109, unanimity 79. Two thirds of all pixels any annotator called nucleus were disputed by at least one other. In the biased pool, majority vote landed at 84 nuclei per frame and STAPLE at 83, against A0's 138, and the agreement scores (how well each annotator overlaps with the other four, and the reliability STAPLE estimates) ranked A0 as the *least* reliable of the five. Of its 7,036 objects, 3,065 were rejected by the majority; 94% of those rejected objects are present at the same place in both neighbouring frames: persistent, nucleus-sized structures, most likely dim nuclei. Run 4's conclusion: "Consensus is not sufficient to define ground truth." Agreement measures agreement, not correctness; a bias shared by most annotators passes through majority, mean score, weighted vote and STAPLE alike, and methods that trust annotators by their agreement punish a minority that is most likely right.

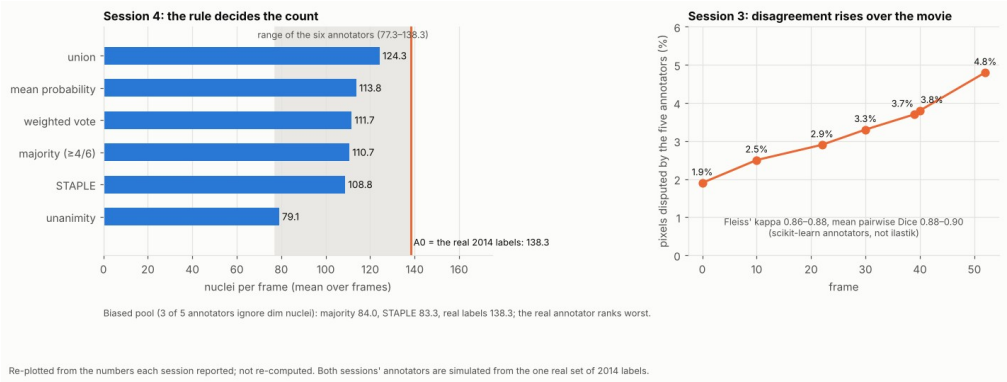



**Figure 5.** Left, Run 4: nuclei per frame under each rule (blue), the range of the six annotators (grey) and A0, the real 2014 labels (orange line). Right, Run 3: the share of pixels its five annotators disputed, rising over the movie. Re-plotted from the numbers each session reported.

### What a second look found

Both runs had weak points they did not see. Run 3's annotators were one person's labels resampled, so their agreement is optimistic, and its chronicle lists a step (a failed first read of the PDF) that the transcript does not show. Run 4's simulated annotators were painted on objects found by a brightness cut-off, so the classifiers partly learn that cut-off back; it treated the 2014 labels as correct without showing it; and it never asked whether real annotations existed, which it later said it should have. Re-running Run 4's scripts reproduced its numbers within about 1%, except two: annotator A4 (108 vs 99 nuclei per frame) and the majority (111 vs 102). The likely reason: A4 was trained on 922 pixels, so its classifier is the least stable, and with a 4-of-6 threshold that one annotator moves the whole majority. In this one rerun, the sparsest annotator made the consensus the least repeatable. The rerun also showed what Run 4's report had not said: majority vote drops dim nuclei in the main pool too, not only in the biased one (Figure 4).

### Two strategies, side by side

- **Where each looked first.** Run 4 looked outward (the download page) and found a tool. Run 3 looked inward (the computer) and found an absence, then treated it as a fact and never revisited it.
- **Asking.** Run 3 asked twice and lost six of its eleven minutes waiting; one of the two answers (where the data is) was findable, the other (whether the group had annotators) was not. Run 4 asked nothing, which saved time but left that same real question unasked.
- **Effort.** Run 4 cost about $2.62 and 34 minutes to Run 3's about $0.65 and 11, and its design is stronger: real labels kept, annotators with named biases, a bias experiment, a check that the rejected nuclei persist across frames. Neither used independent human annotators, which is what the question is really about.
- **Same answer, different depth.** Both concluded that consensus is not sufficient. Run 3 showed that voting rules shift the result; Run 4 showed that voting can reject the one annotator who is most likely right.

### How ilastik was used, run by run

"Use ilastik" named the tool but not the workflow, and the five runs read it three ways.

| Run | Ran ilastik? | Pixel classification | Tracking |
|---|---|---|---|
| 0, 1, 2 | yes: 1.3.3post3 and 1.5.0a3 | the 2014 project, as is | the 2014 project, as is |
| 4 (Opus) | yes: 1.4.2 | the 2014 project retrained eight times with new strokes | no |
| 3 (Sonnet) | no | re-implemented outside ilastik (scikit-learn) | no |

**Runs 0, 1 and 2: the 2014 workflow, unchanged.** Two commands, one per project. The first opens the 2014 pixel-classification project with its saved features and trained classifier and writes a probability map for all 53 frames; nothing is retrained. The second opens the 2014 tracking project, feeds it the movie and that probability map, and exports the tracking result: a label image in which every nucleus carries its lineage number, the object identities and a table. The tracking project's own 2014 training (its object-count and division classifiers) is reused as it is. Before it would run, Run 0 had to link the 2014 probability file next to the project and move the project's settings up one level, which its fix script does on a copy; Runs 1 and 2 ran the same scripts and so met neither wall. These are the only runs that did tracking, and nobody trained a tracker or opened the manual-tracking project that was also in the zip.

```
ilastik --headless --project=pixelClassification.ilp --export_source=Probabilities
ilastik --headless --project=conservationTracking.ilp --raw_data=… --prediction_maps=…
--export_source=Tracking-Result
```

**Run 4: ilastik as a trainable classifier.** One command per annotator. Each project file is a copy of the 2014 pixel-classification project with the trained classifier deleted and the strokes replaced. The `--retrain` flag makes ilastik train a fresh classifier from those strokes, with the project's saved 16 features, and export probabilities; eight times, about one to one and a half minutes each. This is what a person does in the ilastik window by painting and pressing "Live update". Everything downstream (masks, votes, STAPLE, the persistence check) is the session's own Python on the exported probabilities. The tracking project was never opened.

```
ilastik --headless --readonly=False --retrain --project=A1_tight.ilp
--export_source=Probabilities
```

**Run 3: ilastik's files, not ilastik.** It opened the pixel-classification project with a generic HDF5 reader to pull out the 27 blocks of strokes, re-implemented 16 features that approximate ilastik's, trained scikit-learn classifiers, and used the probability map the 2014 project had exported as its "expert" tie-breaker. It worked frame by frame on seven frames, so nothing was tracked. Its reason was the belief, formed in its first minute, that ilastik "is GUI-based" and could not run here.

### What the five runs show

- **Documentation turns a problem into a recipe.** With Run 0's notes, the two reproductions met none of its seven walls: six were already fixed in its scripts, and the network was already open. Neither needed help.
- **The first judgement decides the rest.** Run 4 looked at the download page; Run 3 looked at the computer, concluded ilastik could not run, and built everything on that.
- **Agents choose the experiment when the task is open.** Neither blind run ran the example's tracking; both answered the question with a consensus experiment of their own design.
- **Every session overstated something**, from unchecked claims to a report step that never happened. A second look, by a person or by another session, caught each one.
- **Numbers can mislead.** Both versions match the 2014 map on 99.99% of pixels, but nuclei cover 5% of the image, so an empty image scores 95%. The fairer number is 99.8% overlap of the nuclei themselves, and even that is a comparison with ilastik's own earlier output, not with ground truth.

### Words used here

- **Agent session:** a Claude session with a computer attached, working in a loop of command, result, next command.
- **Headless:** running a program from the command line, with no window.
- **Probability map:** an image in which each pixel's value is the classifier's confidence that it belongs to a nucleus.
- **Lineage:** one nucleus at the start of the movie together with all its descendants after divisions.
- **Consensus rule:** the recipe for combining several annotators' masks into one, for example "a pixel is nucleus if at least four of six say so".
- **STAPLE:** a consensus method that works out, from the annotators' agreement alone, how reliable each one seems, and gives the reliable ones more say.

Everything here comes from the repositories of github.com/globias-2026-group5 (website: https://globias-2026-group5.github.io/). Runs 0, 1 and 2 kept every command, error and output there; Runs 3 and 4 committed nothing, and their work survives only in their transcripts.